AI Detective

A mystery game where a language model writes the case and plays the suspects

Janhavi Bhattad
Reg. No. 235890482
Activity-Based Assessment: Generative AI in Gaming
October 2026

1. The game

AI Detective is a browser-based mystery game. The player chooses a setting (a Bollywood film set, an IPL final, a royal wedding, a Mars base, or any idea they type in) and a crime: murder, theft, disappearance or sabotage. A language model then writes a complete case on the spot, with four suspects, four locations to search, a set of clues and a hidden solution. The player questions the suspects in their own words, searches locations, collects evidence and finally names the culprit. Because the case is written fresh each time, no two games are the same.

I picked the mystery concept because it gives the model several real jobs inside the game. It is not used to write the code. It writes the story, speaks as every character, and reacts to what the player does.

2. How it is played

The player has a limited number of turns: 15 on Easy, 12 on Normal and 8 on Hard. Questioning a suspect, searching a location and asking the partner for a hint each cost one turn. Accusing someone costs nothing but ends the game.

Each suspect is played by the model. Innocent suspects tell the truth about the crime, although they may hide a personal secret that has nothing to do with it. The culprit lies and sticks to an alibi. When asking a question the player can also show the suspect an exhibit from the notebook. An innocent suspect explains it honestly, while the culprit gives an excuse that does not quite fit. Catching that is the main skill in the game.

A few rules make the case push back. They are ordinary Python in *game.py*, not AI. Every suspect has a nerve meter that rises when shown evidence that really points at them, and after two pieces they crack and their secret becomes an exhibit. The room holding the most evidence starts sealed until the player has logged two pieces of evidence. A witness comes forward a third of the way in. Halfway through, the culprit tries to destroy evidence in a room the player has not searched, and the player has two turns to get there first.

To accuse, the player fills in a deduction board: the culprit, the motive and the exhibit that proves it, plus an optional explanation in their own words. A correct accusation scores 100, plus 40 for the motive, 40 for the key evidence, 10 per unused turn and 20 if no hint was used. A wrong accusation scores 0. The player earns up to three stars and a rank of Constable, Inspector or Master Detective.

3. Where generative AI is used

All model calls are in *ai.py*. The game uses Groq's API with the openai/gpt-oss-120b model. The model generates four things, all while the game is running, and every piece of AI text is labelled "AI-written" on screen.

Element	What the model is given	What it produces
The case	The setting, the crime type and a strict list of rules: four suspects, exactly one culprit, at least two clues that break the culprit's alibi, one or two red herrings.	The whole case as JSON: title, victim, suspects, locations, clues and solution.
Suspect answers	That suspect's personality, alibi and secret, whether they are guilty, the interview so far, the clues the player has found and any exhibit being shown.	A short answer in that character's voice, streamed word by word.
Partner hints	The real solution (marked secret), what the player has found and what is still unsearched.	One hint that points the player somewhere useful without naming the culprit.
The ending	Who was accused, whether it was right, the true solution and the player's reasoning.	The final reveal scene and a grade for the reasoning: Sharp, Lucky or Off track.

The suspect prompt is the part that makes the game work. It includes the clues the player has already found, so a suspect can react to a specific piece of evidence instead of repeating the same alibi. The player's choices decide what the model is asked, which is how the game responds to the player rather than just replaying text.

4. Checking what the model writes

A model can return a broken case, for example two culprits or a clue in a room that does not exist. *validate_case* in *game.py* checks every generated case before the game starts. It requires three to five suspects with unique names, exactly one culprit, at least three locations, at least four clues that sit in real locations, and at least two clues pointing at the culprit. If the case fails, the game asks the model once more. If it fails twice, the player sees a message and can retry or play a backup case.

The game rules are kept out of the model on purpose. Turns, scoring and clue discovery are plain code, so the model cannot change the score or the number of turns left.

5. Error handling

- Empty questions and questions over 300 characters are refused with a short message, and no turn is used.
- A location cannot be searched twice, no action works once the turns are used up (except accusing), and the player has to tick a confirmation box before the final accusation.
- A missing or wrong API key, a rate limit or a lost connection produces a plain explanation instead of an error screen. The Groq client retries network errors and rate limits a couple of times first.
- A turn is only spent after the model has answered, so a failed call costs the player

nothing.

- If the model cannot be reached at all, the player can start one of three hand-written backup cases (a murder, a theft and a sabotage). These are labelled "Hand-written". If only the ending fails, the plain solution is shown.
- The API key is read from a *.env* file that git ignores. The repository contains only *.env.example*.

6. A recorded run

This is a short extract from a real game against the live model, using the royal wedding setting and the theft crime. The case summary is condensed; the search result, the two answers and the hint are word for word.

Case written by the model: The Crown Jewel Heist. A sapphire diadem has vanished from the palace vault during the wedding. Suspects: Lady Vivienne (Lord Marlowe's estranged sister), Captain Rowan (head of palace security), Mira (royal florist) and Sir Tobias (chief steward). Locations: Vault, East Courtyard, West Pavilion, Garden Gazebo.

Search, East Courtyard: "A set of fresh footprints leads from the fireworks launch point to the vault's side entrance, but stop before the door."

Question to Captain Rowan: Where were you when it happened?
Answer: I was on the East Courtyard, sweeping the perimeter as the fireworks ignited. My sector covers the launch zone, so I kept watch there until the display peaked.

Same suspect, shown the footprints exhibit: Can you explain this?
Answer: Those prints are from the night-vision crew that set up the pyrotechnics; they moved back to the platform after the launch. I never crossed that path, and the vault door was still sealed when my patrol turned back toward the East Courtyard.

Partner hint: Check the spare-key lock on the vault's side entrance. Its recent tampering doesn't match the official patrol logs and may reveal who actually opened it.

Captain Rowan was the culprit in this case. His first answer is a smooth alibi. Once shown the footprints, he replies with an explanation that blames someone else, which is exactly the kind of excuse the player is meant to question.

7. Tools and how to run

Python 3, Streamlit for the interface, the Groq API with openai/gpt-oss-120b, python-dotenv for the key, and unittest with Streamlit's AppTest for tests. I first planned to use Llama 3.3, but Groq had retired it, so I moved to GPT-OSS 120B. That model spends part of its reply length on hidden reasoning, which at first cut off the end of the case JSON. Setting its reasoning effort to low fixed this. The model name can be changed with *GROQ_MODEL* in *.env*.

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env (then paste a Groq key into .env)
streamlit run app.py
```

The game opens at http://localhost:8501. A free key can be created at console.groq.com/keys.

8. Testing

There are 23 automated tests, run with *python -m unittest*. They cover the rules, scoring, the question checks, the case validator, the retry on bad JSON, the missing-key message, all three backup cases and one complete game played through the real interface from the start screen to a correct accusation. The model is replaced with fixed replies in the tests, so they need no key and finish in under a second. I also played full games against the live model, including a wrong accusation and the ending screen. While preparing this report I found that the case checker would accept a case whose locations were plain strings, which then crashed the game when it started. The checker now rejects that and the game asks the model again. A test covers it.

9. Limits

The suspects are written by a language model, so an answer can occasionally contradict a clue or drift out of character. The prompt tells suspects to stay in character and never reveal the culprit, but this is not guaranteed. A game needs an internet connection and a Groq key, apart from the backup cases, which still use the model to play the suspects. Game history is stored in a local file, so it is not shared between players.